CSC 370: Databases 4NF & Multivalued Dependencies

Sean Chester • University of Victoria

Which are in 3NF decomposition?

R(A,B,C,D)

$A \rightarrow BC$ $B \rightarrow A$ $AB \rightarrow C$

- ❏ R(A, B)
- ❏ T(B, C)
- ❏ S(A, C)
- ❏ U(A, D)

Which are in 3NF decomposition?

R(A,B,C,D)

$A \rightarrow BC$ $B \rightarrow A$ $AB \rightarrow C$

- ❏ R(A, B)
- ❏ T(B, C)
- ❏ S(A, C)
- ❏ U(A, D)

Did we really need to split R, S, and T? (Different sources given different answers here)

Intended Learning Outcome:

When examining a relation R, you can:

Correctly identify any multivalued dependencies and decompose R into 4NF

Outline

1. Recall FD's, closures, and decomposition from lens of dependent vs independent attributes
2. Introduce multivalued dependencies (MVDs) as a generalisation of FDs
3. Work through some example relations to establish whether they contain MVDs
4. Work through a decomposition into *fourth normal form* (4NF)

Last class...

Post-mortem

Let's decompose into BCNF the following schema and FD's:

CinemaShowings(city, theatre, title)
theatre → city
city, title → theatre

Showings(theatre, title)
Theatres(theatre, city)

Why did we break apart the key {city, title}?

- Only half of that key appeared in the closure of the violating FD!
- Where did the FD {city, title} → {theatre} go?
- Our BCNF database doesn't enforce the second constraint with key constraints anymore!
    - So much for opportunities to monopolise competition and engage in anti-competitive practices! 😭 (It's the textbook's example, not mine.)

Source: Garcia-Molina et al. (2007), §3.4.4

Prime Attributes

We can recognise the problem from the previous example by identifying *prime* attributes:

- An attribute is *prime* iff it is part of a key
- **Note**: that is a *key*, not a *superkey*!

CinemaShowings(city, theatre, title)
theatre → city
city, title → theatre

Observe that {theatre, title} was the only key. Thus:

- *city* is **not** prime
- *theatre* is prime
- *title* is prime

BCNF decomposition will usually lose constraints if some FD has an antecedent involving a prime attribute that is not a key.

Source: Garcia-Molina et al. (2007), §3.4.4

Third Normal Form (3NF)

A relation R over attributes C with functional dependencies F is in 3NF iff:

Every FD f in F is in 3NF.

An FD f:X→Y is in 3NF iff:

X is a superkey
OR
Every attribute in Y is "prime"

Equivalently:

f is not a BCNF violation
OR
Every attribute in Y is "prime"

Source: Garcia-Molina et al. (2007), §3.5.1

Revisiting the movie example

Let's decompose into **3NF** the following schema and FD's:

R(city, theatre, title)
f_0: theatre → city
f_1: city, title → theatre

Observe that:
$\{city,title\}^+ = \{city, title, theatre\}$
$\{theatre,title\}^+ = \{city, title, theatre\}$

f_1 is not a BCNF violation because its antecedent is a superkey. Therefore it is also in 3NF
f_0 *is* a BCNF violation because $\{theatre\}+ \neq \{city, theatre, title\}$;
- However, city is *prime* (part of key {city,title}), so f_0 is in 3NF

Because this relation is already in 3NF, we do not decompose it and the dependencies are preserved.

Source: Garcia-Molina et al. (2007), §3.5.1

3NF Synthesis Algorithm

Input: A relation R with attribute set C and a set of functional dependencies F
Output: A set of projections of R and F that are in 3NF, have a lossless join, and preserve dependencies

1. Let G be a minimal basis for F
2. For each functional dependency g: $X \rightarrow Y$ in G:
   a. Create a sub-relation by projecting R onto $X \cup Y$
3. If no sub-relation has a superkey for R:
   a. Create a sub-relation with *some* key
4. Return all sub-relations

Source: Garcia-Molina et al. (2007), §3.5.2

Another view of normality

Recall: closures and BCNF

Assume we have:
R(a,b,c,d)
a→b
b→c

How do we begin BCNF decomposition?

We calculate the closures for the left-hand sides of each FD and see if we have superkeys:

$\{a\}^+ = \{a,b,c\} \neq \{a,b,c,d\}$
$\{b\}^+ = \{b,c\} \neq \{a,b,c,d\}$

Thus, both FD's are BCNF violations… And we should decompose based on them.

But let's step back and look at this from another perspective…

Normalisation seen through independence

Assume we have:
R(a,b,c,d)
A={a,b,c,d}
a→b
b→c

Let's create a disjoint partitioning of A based on each minimal basis FD:

FD	Join key (J)	Dependent D= ($J^+ \setminus J$)	Independent I = ($A \setminus J^+$)
a→b	a	b,c	d
b→c	b	c	a,d

Normalisation seen through independence

Assume we have:
R(a,b,c,d)
A={a,b,c,d}
a→b
b→c

Let's create a disjoint partitioning of A based on each minimal basis FD:

FD	Join key (J)	Dependent D= ($J^+ \setminus J$)	Independent I = ($A \setminus J^+$)
a→b	a	b,c	d
b→c	b	c	a,d

Consider what BCNF Decomposition does in each case:

a→b: R1(a,b,c); R2(a,d) R1(J,D); R2(J, I)

b→c: R1(b,c); R2(b,a,d) - We pick some FD and split determined / independent attributes
- We add the join key (left-hand side) to both so it's not lossless

Normalisation seen through independence

Assume we have:
R(a,b,c,d)
A={a,b,c,d}
a→b
b→c

Let's create a disjoint partitioning of A based on each minimal basis FD:

FD	Join key (J)	Dependent D= ($J^+ \setminus J$)	Independent I = ($A \setminus J^+$)
a→b	a	b,c	d
b→c	b	c	a,d

Consider what BCNF/3NF Decomposition does in each case:

- J_i is a superkey iff I_i is the empty set
- J_i is a key iff I_i is the empty set *and* no proper subset of $J_j \subset J_i$ is a superkey
- FD_i is a BCNF violation (i.e., should be split) if I_i is non-empty
- FD_i is a 3NF violation if I_i is non-empty and D_i contains a non-prime attribute

Normalisation seen through independence

Assume we have:
R(a,b,c,d)
A={a,b,c,d}
a→b
b→c

Let's create a disjoint partitioning of A based on each minimal basis FD:

FD	Join key (J)	Dependent D= ($J^+ \setminus J$)	Independent I = ($A \setminus J^+$)
a→b	a	b,c	d
b→c	b	c	a,d

But why? What is so important about the column I?

- Observe that if I_i and D_i are both non-empty, we need to track all combinations of them wrt J_i
- This is the very reason for normalisation!

Recall: Relation Anomalies

v_number	name	class
V00111222	Alice	CSC 370
V00111222	Alice	CSC 320
V00333444	Bob	CSC 370

v_number→name

J: {v_number}
D: {name}
I: {class}

For V00111222:
- 1 name × 2 classes

For V00333444:
- 1 name × 1 class

There is only 1 name, but it is combined w/ every class;
After decomposition, it only appears once.

Redundancy: Information is unnecessarily repeated; in this case Alice's name appears twice

Update Anomaly: If we change some information we may lose consistency in the data. For example, if Alice changes her name to 'Carol' we may forget to replace both instances. (It is possible to make this impossible.)

Deletion Anomaly: If some information is deleted (e.g., Bob drops CSC 370), we may lose the rest of the information (Bob's name and v-number).

We shouldn't solve poor design with NULLs.

Multivalued Dependencies (MVDs)

Generalisation to MVD's

v_number	name	fav_class
V00111222	Alice	CSC 370
V00111222	Alice	CSC 320
V00333444	Bob	CSC 370
V00333444	Robert	CSC 370

v_number→name?

J: {v_number}
D: {name}
I: {class}

For V00111222:
- 1 name × 2 classes

For V00333444:
- 2 names × 1 class

Now x names are combined w/ every class;
After decomposition, it only appears once.

What if names were also set-valued?
- We still know all the names, given a v_number, but it's not an FD
  - Two rows have the same v_number but different names

So what happens now if student V00333444 also really likes CSC 320?

Generalisation to MVD's

v_number	name	fav_class
V00111222	Alice	CSC 370
V00111222	Alice	CSC 320
V00333444	Bob	CSC 370
V00333444	Robert	CSC 370
V00333444	Robert	CSC 320
V00333444	Bob	CSC 320

We need to add extra rows to track all the name–fav_class associations. Clearly, there is an anomaly here!

What if names were also set-valued?

- We still know all the names, given a v_number, but it's not an FD
  - Two rows have the same v_number but different names

So what happens now if student V00333444 also really likes CSC 320?

v_number→name?

J: {v_number}
D: {name}
I: {class}

For V00111222:

- 1 name × 2 classes

For V00333444:

- 2 names × 2 classes

Now x names are combined w/ every class; After decomposition, it only appears once.

Generalisation to MVD's

v_number	name	fav_class
V00111222	Alice	CSC 370
V00111222	Alice	CSC 320
V00333444	Bob	CSC 370
V00333444	Robert	CSC 370
V00333444	Robert	CSC 320
V00333444	Bob	CSC 320

We need to add extra rows to track all the name–fav_class associations. Clearly, there is an anomaly here!

v_number↠name

J: {v_number}
D: {name}
I: {class}

- We still have a partitioning on v_number into attributes determined from it and those that are independent from it
- However, what v_number determines is a set ("multi values")!
- We call this a *multivalued dependency*
- (Actually, an FD determined a set value too, but that was *always* from a singleton set)

Multivalued Dependencies (MVD's) - Formally

Given a relation R with attribute set A:

- Let $B \subseteq A$ and $C \subseteq A$

We have a multivalued dependency $B \twoheadrightarrow C$, i.e., B *multidetermines* C, iff:

For all tuples $t,u \in R$ where $\pi_B(t) = \pi_B(u)$

There exists a third tuple $v \in R$ such that:

- $\pi_B(t) = \pi_B(u) = \pi_B(v)$
- $\pi_C(t) = \pi_C(v)$
- $\pi_{A \setminus B \setminus C}(u) = \pi_{A \setminus B \setminus C}(v)$

Informally, we have that:

- C is the set of attributes that are dependent on B and $A \setminus B \setminus C$ is independent of those

Identifying MVDs

MVD Example 2

Can we identify an MVD below?

Course	Text Title	Text Author	CRN	Instructor
CSC 370	“Database Systems”	Ullman et al.	12345	Chester
CSC 370	“MongoDB Basics”	Membrey et al.	12345	Chester
CSC 370	“Database Systems”	Ullman et al.	23456	Thomo
CSC 370	“MongoDB Basics”	Membrey et al.	34567	German

MVD Example 2

Can we identify an MVD below?

Course	Text Title	Text Author	CRN	Instructor
CSC 370	"Database Systems"	Ullman et al.	12345	Chester
CSC 370	"MongoDB Basics"	Membrey et al.	12345	Chester
CSC 370	"Database Systems"	Ullman et al.	23456	Thomo
CSC 370	"MongoDB Basics"	Membrey et al.	34567	German

How about CRN ↠ Course Instructor?

- Pick any two tuples with the same CRN and call one of them *t* and the other *u*. For example:
  - Let *t* be the second tuple and *u* be the one above it
  - Then there must be a tuple *v* = (CSC 370, "MongoDB Basics", Membrey et al., 12345, Chester)
    - Well, actually, that is just *t*. So, yes! But this MVD was just an FD…

FD's vs MVD's

Observe that every FD is an MVD

Let a→b be an FD for R(a,b,c):

Let us consider the definition of an MVD:

If we have one tuple $t_i = (a, b_i, c_i)$
+ we have one tuple $t_j = (a, b_j, c_j)$

Then, an MVD requires that we can find a tuple $t_k = (a, b_i, c_j)$

But, since a→b, and both tuples have the same a value, we can assert that $b_i = b_j$.
Therefore, t_k certainly exists, it is always t_j!

MVD Example 3

Can we identify an MVD below?

Course	Text Title	Text Author	Semester	Instructor
CSC 370	“Database Systems”	Ullman et al.	Winter 2022	Chester
CSC 370	“MongoDB Basics”	Membrey et al.	Winter 2022	Chester
CSC 370	“Database Systems”	Ullman et al.	Fall 2022	Thomo
CSC 370	“MongoDB Basics”	Membrey et al.	Summer 2022	Chester
CSC 370	“Database Systems”	Ullman et al.	Summer 2022	Chester

MVD Example 3

Can we identify an MVD below?

Course	Text Title	Text Author	Semester	Instructor
CSC 370	"Database Systems"	Ullman et al.	Winter 2022	Chester
CSC 370	"MongoDB Basics"	Membrey et al.	Winter 2022	Chester
CSC 370	"Database Systems"	Ullman et al.	Fall 2022	Thomo
CSC 370	"MongoDB Basics"	Membrey et al.	Summer 2022	Chester
CSC 370	"Database Systems"	Ullman et al.	Summer 2022	Chester

How about Course Instructor ↠ Text_Title Text_Author?

- Pick any two tuples with the same Course and Instructor:
  - Let *t* be the fourth tuple and *u* be the second tuple
  - Then there must be a tuple *v* = (CSC 370, "MongoDB Basics", Membrey et al., Winter 2022, Chester)
    - Indeed, that's the first tuple!

MVD Anomalies

Course Instructor ↠ Text_Title Text_Author

Course	Text Title	Text Author	Semester	Instructor
CSC 370	“Database Systems”	Ullman et al.	Winter 2022	Chester
CSC 370	“MongoDB Basics”	Membrey et al.	Winter 2022	Chester
CSC 370	“Database Systems”	Ullman et al.	Fall 2022	Thomo
CSC 370	“MongoDB Basics”	Membrey et al.	Summer 2022	Chester
CSC 370	“Database Systems”	Ullman et al.	Summer 2022	Chester

So, if Dr. Chester always uses two texts for CSC 370, what happens if he teaches another semester?

- We have to add two extra tuples that differ only in one attribute — a lot of redundancy!

It’s better for this database to ask Dr. Thomo instead! 😂

Working with MVDs

Rules on MVD's: triviality

Triviality
Let C be the attributes of a relation R and A be any subset of C. Then we have both:
A↠C\A
A↠A

The first case is especially important.

Consider the definition:

- For any tuples *t* and *u*, there must be a tuple *v* with *u*'s RHS values and everything else from *t*
  - In the first case "everything else" (the independent variables) is empty; so, obviously this tuple *v* is just *u*
  - In the second case, the RHS is effectively empty; so, obviously this tuple *v* is just *t*.

By now, you should be noticing a sort of symmetry…

Rules on MVD's: Complementarity

Complementarity
Let C be the attributes of a relation R and A and B be a subset of B. Then we have:
$A \twoheadrightarrow B \Rightarrow A \twoheadrightarrow C \backslash B$

Consider the definition again:

- For any tuples *t* and *u*, there must be a tuple *v* with *u*'s RHS values and everything else from *t*
  - Clearly, we can just swap *t* and *u* because the rule applies for all pairs of tuples that match on the join key
  - Stated differently, in MVD's (that are not FD's), there is no meaningful difference between the set-valued group of attributes that can be determined from a join key and the set of values that are taken for the other attributes

Importantly, observe that this rule only weakly applies to FD's. Given $A \rightarrow B$:

- We have that $A \twoheadrightarrow C \backslash B$
- *But not necessarily that:* $A \rightarrow C \backslash B$

Rules on MVD's: transitivity and splitting

Transitivity
Let C be the attributes of a relation R and A be any subset of C.
If A↠B and B↠C, then:
A↠C

Splitting Rule
If a↠bc:

*We cannot split it into (*Because of the complementarity rule*):*
A↠b
A↠c

Fourth Normal Form (4NF)

Fourth Normal Form

We say that a relation R with attributes C and MVD's F is in 4th Normal Form (4NF)

If and only if:

For every non-trivial MVD $A \twoheadrightarrow B$, A is a superkey.

Observe the similarity to BCNF.

4NF Decomposition (relation R, attributes C, MVD's F):

1. If there are no 4NF violations, return R
2. Select a 4NF violation, $A \twoheadrightarrow B$
3. Split R into $A \cup B$ and $(C \setminus B) \cup A$
4. Project F onto each sub-relation
5. Return the union of the recursive calls on each sub-relation

This will separate attribute partitions D and I into separate relations, just like in BCNF.
Observe that to generalise BCNF, we must assume every FD is written to determine its entire closure

4NF Decomposition Example

Let’s identify constraints for the following example:

MovieShowings(title, year, genre, theatre, city, imdbRating)

4NF Decomposition Example

Let's identify constraints for the following example:

MovieShowings(title, year, genre, theatre, city, imdbRating)

We have MVD's:

1. title year ↠ genre
2. theatre → city
3. title year → imdbRating

We see (1) is a non-trivial MVD and {title, year} is not a superkey. So, we decompose it:
Genres(title, year, genre)
Showings(title, year, theatre, city, imdbRating)

But both Showings has a 4NF violation because:

- title year → imdbRating, but {title, year} is not a superkey for Showings
- theatre → city, but {theatre} is not a superkey for Showings

4NF Decomposition Example

Let's identify constraints for the following example:

MovieShowings(title, year, genre, theatre, city, imdbRating)

We have MVD's:
title year ↠ genre theatre → city title year → imdbRating

Genres(title, year, genre)
Showings(title, year, theatre, city, imdbRating)

But both Showings has a 4NF violation because:
- title year → imdbRating, but {title, year} is not a superkey for Showings
- theatre → city, but {theatre} is not a superkey for Showings

So we decompose it to get:
Movies(title, year, imdbRating)
Theatres(theatre, city)
CinemaShowings(theatre, title, year)

4NF Decomposition Example

Let's identify constraints for the following example:

MovieShowings(title, year, genre, theatre, city, imdbRating)

How did this differ from BCNF?

With the extra MVD
Genres(title, year, genre)
Movies(title, year, imdbRating)
Theatres(theatre, city)
CinemaShowings(theatre, title, year)

Without the extra MVD
Movies(title, year, imdbRating)
Theatres(theatre, city)
CinemaShowings(theatre, title, year, genre)

With the extra MVD, we don't have to pair every showing with every genre for each movie

Summary / Exercise

Normal Forms

There are several other normal forms that we have not looked at:

- 4NF considers multivalued dependencies
- We can construct a containment hierarchy of the different normal forms

*This hierarchy is clear because 3NF is like BCNF **OR** another condition whereas 4NF is like BCNF **AND** another type of constraint*

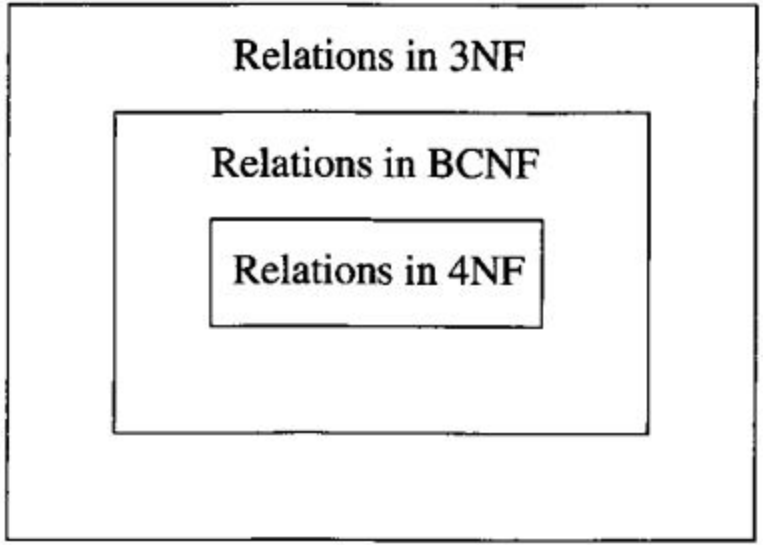


Figure 3.12: 4NF implies BCNF implies 3NF

Source: Garcia-Molina et al. (2007), §3.6.6

Take-home Exercise

Provide another example of a relation that contains an MVD